САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

ОТЧЁТ

по домашней работе № 5

Выполнил:
Цатинян Артём
Группа K3439

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Цель работы

Изучить основные команды npm и создание приложения на Vue.js, применить компоненты, роутер, хранилище состояния и взаимодействие с API.

Задание

На основе мануала преподавателя создать приложение заметок на Vue.js. Организовать получение списка и добавление заметок через имитацию API; представить код приложения и скриншоты компонентов.

Ход работы

1. Подготовка проекта и npm

Создан отдельный проект hw5 с использованием Vue 3 и сборщика Vite. В package.json перечислены зависимости приложения: Vue, Vue Router, Pinia, axios и Bootstrap. JSON Server применяется для локального API, Vitest — для проверки хранилища, Playwright — для проверки пользовательского сценария в браузере.

Команда npm install устанавливает зависимости, npm run api запускает JSON Server на порту 3003, а npm start запускает приложение на порту 8080. Для API и приложения используются два терминала. Команда npm run build создаёт production-сборку; npm test выполняет тесты хранилища. Файл package-lock.json фиксирует установленные версии зависимостей.

2. Структура приложения

Файл src/main.js подключает Vue, Pinia, плагин сохранения состояния, роутер и стили Bootstrap. App.vue выводит текущее представление через RouterView. Маршрут / связан со страницей NotesPage.vue. Компонент BaseLayout.vue задаёт контейнер страницы через slot, а NoteCard.vue отображает отдельную заметку.

3. Получение и создание заметок

Общий экземпляр axios настроен в src/api/instance.js с базовым адресом /api. Vite перенаправляет запросы к локальному JSON Server. Методы в src/api/notes.js получают список заметок и создают новую запись. При первом запуске API файл db.json создаётся из db.example.json; рабочий db.json исключён из Git.

src\api\instance.js

```
import axios from "axios";

export const api = axios.create({ baseURL: "/api", timeout: 10000 });
```

src\api\notes.js

```
import { api } from "./instance";

export const notesApi = {
  async getAll() {
    return (
      await api.get("/notes", { params: { _sort: "id", _order: "desc" } })
    ).data;
  },
  async create(note) {
    return (await api.post("/notes", note)).data;
  },
};
```

Хранилище Pinia содержит список заметок, черновик формы, состояния загрузки и сохранения, а также сообщение об ошибке. Действие loadNotes получает список, createNote отправляет запись на сервер и добавляет полученный результат в начало списка. С помощью pinia-plugin-persistedstate сохраняется только черновик формы; сами заметки загружаются с сервера.

src\stores\notes.js

```
import { defineStore } from "pinia";
import { notesApi } from "../api/notes";

export const useNotesStore = defineStore("notes", {
  state: () => ({
    notes: [],
    draft: { name: "", text: "" },
    loading: false,
    saving: false,
    error: "",
  }),
  persist: { pick: ["draft"] },
  actions: {
    async loadNotes() {
      this.loading = true;
      this.error = "";
      try {
        this.notes = await notesApi.getAll();
      } catch {
        this.error =
          "Не удалось загрузить заметки. Проверьте, что сервер API запущен.";
      } finally {
        this.loading = false;
      }
    },
    async createNote(note) {
      this.saving = true;
      this.error = "";
      try {
        const created = await notesApi.create(note);
        this.notes.unshift(created);
        return true;
      } catch {
        this.error = "Не удалось сохранить заметку. Попробуйте ещё раз.";
        return false;
      } finally {
        this.saving = false;
      }
    },
  },
});
```

4. Компоненты и форма

BaseLayout использует slot для размещения содержимого страницы. NoteCard получает название и текст через props, а при отсутствии текста показывает запасное сообщение. В NotesPage поля связаны с данными через v-model, отправка обрабатывается через @submit.prevent, а список карточек выводится директивой v-for. При монтировании страницы вызывается загрузка данных.

После успешной отправки форма очищается. Если сервер недоступен, интерфейс показывает сообщение об ошибке, а введённые данные сохраняются.

src\layouts\BaseLayout.vue

```
<template>
  <main class="container py-5">
    <slot />
  </main>
</template>
```

src\components\NoteCard.vue

```
<script setup>
defineProps({
  name: { type: String, required: true },
  text: { type: String, default: "" },
});
</script>

<template>
  <article class="card h-100">
    <div class="card-body">
      <h2 class="card-title h5">{{ name }}</h2>
      <p v-if="text" class="card-text">{{ text }}</p>
      <p v-else class="card-text text-secondary">Без описания</p>
    </div>
  </article>
</template>
```

src\views\NotesPage.vue

```
<script setup>
import { onMounted, ref } from "vue";
import { storeToRefs } from "pinia";
import BaseLayout from "../layouts/BaseLayout.vue";
import NoteCard from "../components/NoteCard.vue";
import { useNotesStore } from "../stores/notes";

const store = useNotesStore();
const { notes, draft: form, loading, saving, error } = storeToRefs(store);
const formError = ref("");

async function createCard() {
  formError.value = "";
  const name = form.value.name.trim();
  if (!name) {
    formError.value = "Введите название заметки.";
    return;
  }

  const created = await store.createNote({
    name,
    text: form.value.text.trim(),
    userId: 1,
  });

  if (created) {
    form.value.name = "";
    form.value.text = "";
  }
}

onMounted(() => store.loadNotes());
</script>

<template>
  <BaseLayout>
    <h1 class="mb-4">Мои заметки</h1>
```

```
    <form class="mb-5" @submit.prevent="createCard">
      <div class="mb-3">
        <label class="form-label" for="note-name">Название</label>
        <input
          id="note-name"
          v-model="form.name"
          class="form-control"
          maxlength="120"
          required
        />
      </div>

      <div class="mb-3">
        <label class="form-label" for="note-text">Текст заметки</label>
        <textarea
          id="note-text"
          v-model="form.text"
          class="form-control"
          rows="4"
          maxlength="2000"
        ></textarea>
      </div>

      <p v-if="formError" class="text-danger" role="alert">{{ formError }}</p>
      <button class="btn btn-primary" type="submit" :disabled="saving">
        {{ saving ? "Сохранение..." : "Добавить заметку" }}
      </button>
    </form>

    <div v-if="error" class="alert alert-danger" role="alert">
      {{ error }}
      <button class="btn btn-link" type="button" @click="store.loadNotes()">
        Повторить
      </button>
    </div>

    <p v-if="loading" role="status">Загрузка заметок...</p>
    <p v-else-if="!notes.length">Заметок пока нет.</p>
    <div v-else class="row row-cols-1 row-cols-md-2 g-3">
      <div v-for="note in notes" :key="note.id" class="col">
        <NoteCard :name="note.name" :text="note.text" />
      </div>
    </div>
  </BaseLayout>
</template>
```

5. Результат работы

На рисунке 1 показаны загруженные карточки заметок. Далее в форму введены название и текст новой заметки; после отправки новая карточка появилась в списке, а поля формы очистились.

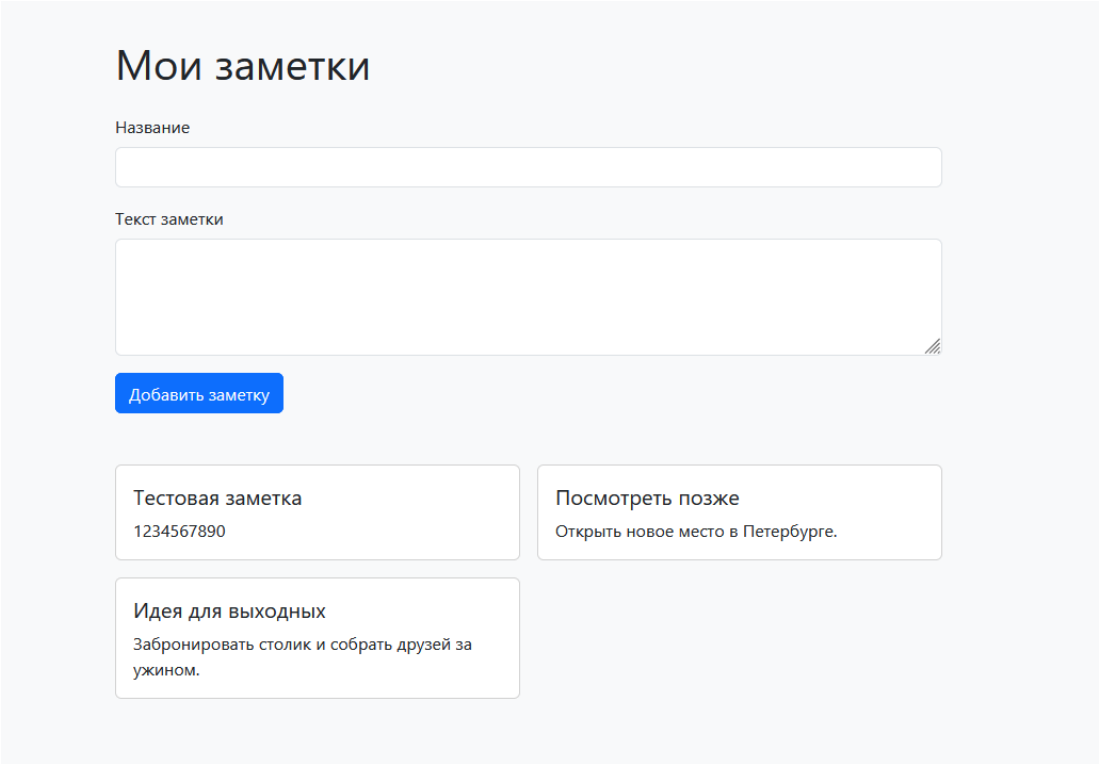


Рисунок 1 — Список заметок, загруженный из API

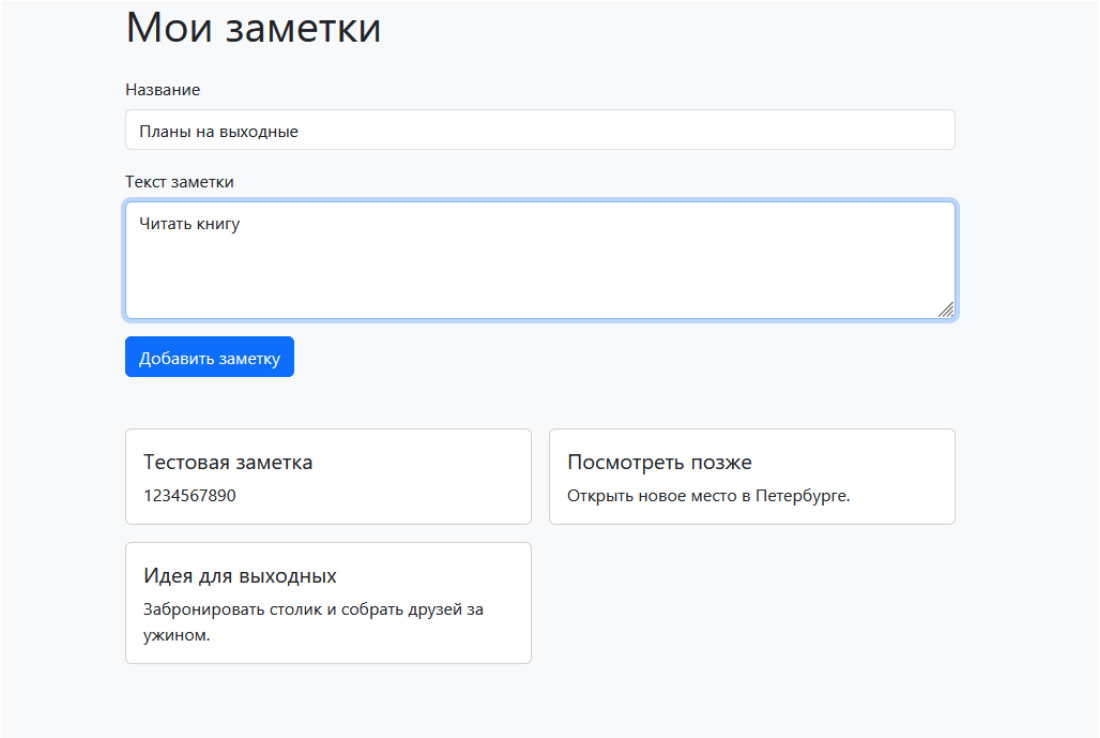


Рисунок 2 — Заполненная форма новой заметки

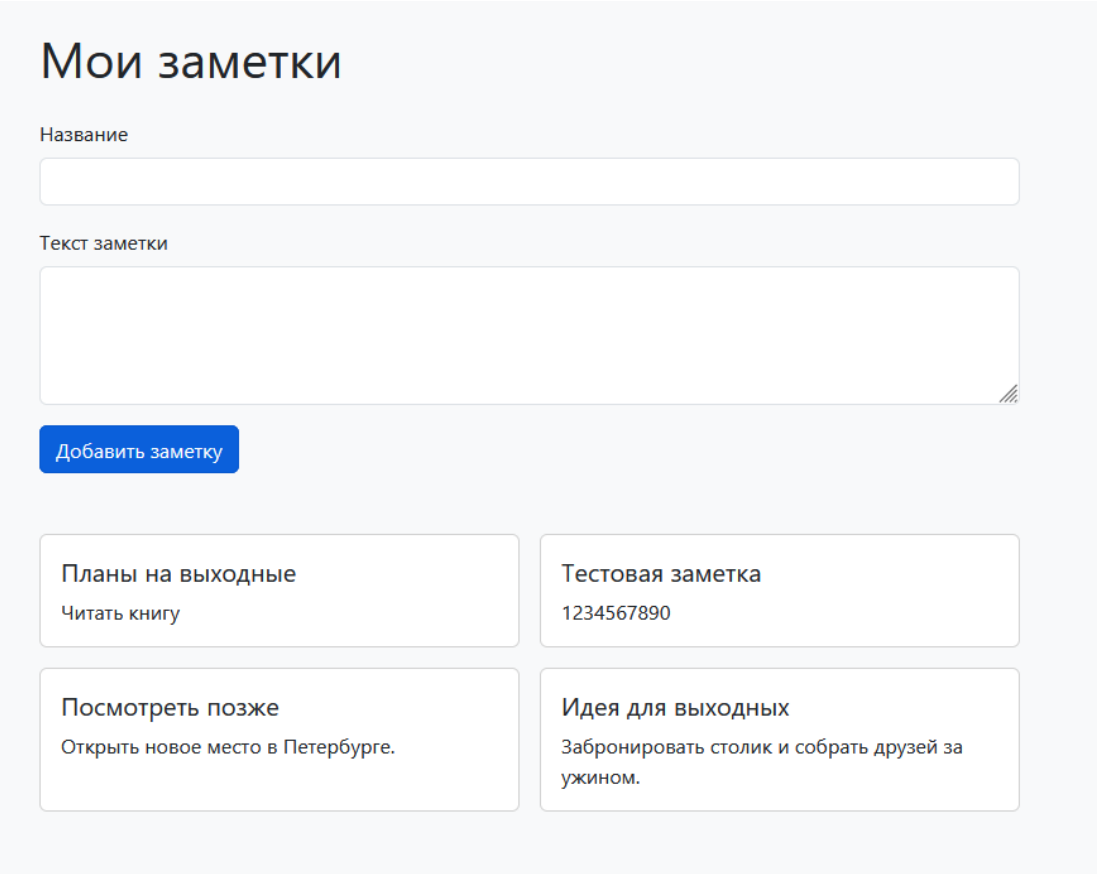


Рисунок 3 — Добавленная заметка и очищенная форма

6. Проверка

Production-сборка приложения выполнена командой npm run build. Три теста Vitest проверяют загрузку заметок, создание записи и обработку ошибки. Браузерный сценарий проверяет сохранение черновика после обновления страницы, добавление заметки, повторную загрузку списка и отображение на мобильной ширине.

Вывод

В результате работы создано приложение заметок на Vue.js. Освоены установка npm-зависимостей и запуск скриптов проекта, разработка компонентов с props и slot, связывание полей через v-model, работа с Pinia и отправка запросов к API с помощью axios.